

实验手册二：Pthreads 多线程程序编写

完成下列两个子任务，形成一个完整的实验报告。（C/C++均可，不限制 src、inc、lib、Makefile 的项目形式）

1. 稠密矩阵-稠密矩阵乘 (GEMM)的多线程实现

(1) 实验目的

- 理解 POSIX 线程（Pthreads）的基本使用方法，包含线程创建、回收、属性、同步等。
- 在串行 GEMM 的基础上，掌握数据并行切分与负载均衡方法，完成 $C=A \times B$ 的 Pthreads 并行实现。
- 评估并行化带来的加速比、并行效率与可扩展性。

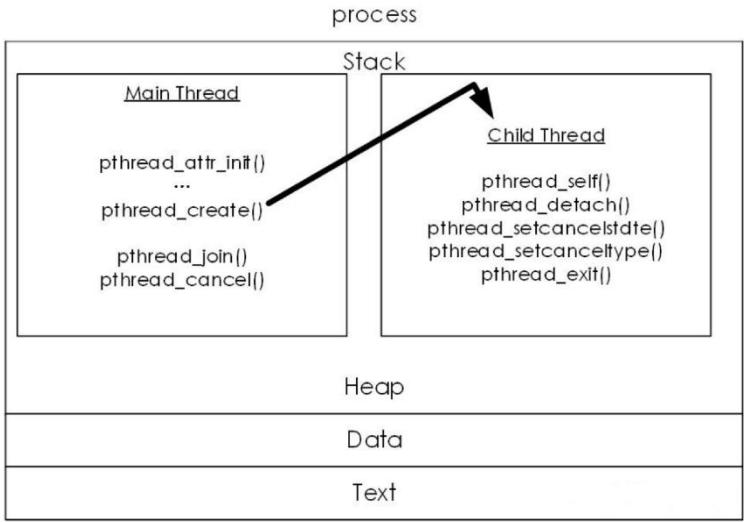
(2) 背景知识（Pthreads 核心要点）

要用 Pthreads 高效地实现 GEMM，我们不能再用单个执行流（单线程）去串行地计算 C 矩阵的每一个元素，核心思想是分而治之，计算 C 矩阵这一宏大任务，拆解成多个可以独立执行的子任务，然后将这些子任务分配给多个线程去并行处理。

为了更好地理解这些线程是如何在一个程序中协作的，我们首先需要清晰地认识 Pthreads 的内存模型。在一个进程（Process）的内存空间中，所有线程共享同一个地址空间，包括代码段（Text）、数据段（Data）和堆（Heap）。这让我们可以在主线程中动态分配的大块内存，比如矩 A、B 和 C，可以被所有子线程直接访问。

然而，每个线程，包括主线程（Main Thread）和它创建的子线程（Child Thread），都拥有自己私有的栈（Stack）。栈用于存放局部变量、函数参数等。这种“全局共享，局部私有”的模型是 Pthreads 编程的基础，我们利用共享的堆来实现数据协同，利用私有的栈来保证执行状态的独立，避免互相干扰。那么下图中的箭头清晰地展示了主线程如何通过 `pthread_create()` 催生出一个拥有独立栈空间的子线程，从而开启并行计算。

理解了这个模型后，我们就可以具体规划如何运用 Pthreads 的功能来实现一个高性能的 GEMM。



在明确了 Pthreads 的内存模型后，实现一个高效的并行 GEMM 需要综合运用下面所提到的关键技术。

首先是并行模式与数据切分，这是我们并行设计的首要步骤，通常采用 Manager/Worker 模型，其中主线程扮演 Manager，负责创建 Worker 线程并划分任务，而子线程则扮演 Worker，负责执行具体的计算任务。任务划分，即数据切分，是性能的关键，那么对于我们要优化的 GEMM，常见的策略有 2 点：

(1) 按行块处理，也就是将结果矩阵 C 的连续若干行打包成一个任务，分配给一个线程，这种方法实现简单，逻辑清晰。

(2) 另一种是 2D 分块，也就是将 C 矩阵划分为多个矩形子块，每个线程负责计算一个或多个子块，这种策略能更好地利用 CPU 缓存的数据局部性，通常性能更优。

另一核心点是线程生命周期管理，管理 Worker 线程的创建与结束是基础操作，**线程创建** (`pthread_create`) 就是 Manager 主线程调用此函数来创建 Worker 线程，那么我们会将任务信息(比如负责的行范围或块索引)通过参数传递给 Worker。**线程回收** (`pthread_join`): 就是主线程必须调用此函数等待所有 Worker 线程完成它们的计算任务，这既是保证计算结果完整性的同步点，也是回收线程资源的必要步骤。

最后是同步原语，同步是确保多线程协作正确性的关键。幸运的是，在设计良好的 GEMM 数据并行实现中，同步开销可以降到最低。我们需要了解无锁计

算：就是如果我们保证每个线程只写入 C 矩阵中预先分配给它的、与其它线程不重叠的区域，那么核心的乘加计算过程是无需加锁的，这极大地提升了并行效率。仅在某些场景下需要同步，例如使用一个全局共享的任务队列进行动态任务分配时，需要互斥量 (`pthread_mutex_*`) 来保护队列的访问，可能需要条件变量 (`pthread_cond_*`) 来高效地唤醒等待任务的 Worker 线程。

最后是调度与亲和性，这是性能优化的进阶手段，旨在减少操作系统调度带来的不确定性。CPU 亲和性 (`pthread_setaffinity_np`) 让我们可以将特定线程绑定到特定的 CPU 核心上，这样做可以减少线程在不同核心间的迁移，从而提高缓存命中率，使性能更稳定、更高。更改调度策略（如 `SCHED_FIFO`）可以给予线程更高的执行优先级，但这需要谨慎使用，通常在实时或对延迟极度敏感的系统中使用较多。

(3) 实验内容与要求

1. 并行实现

- 使用 C/C++ 语言和 Pthreads 编写。
- 数据类型：double。接口： $C = A \times B$ （无需 α 、 β 扩展）
- 切分策略（建议两种均实现，分析优劣）
 - ✧ 按行块：将结果矩阵 C 的行区间划分给不同线程。
 - ✧ 2D 分块：将 C 划为 $\text{tile}(i, j)$ ，每个线程负责一个或多个 tile 的计算。

2. 正确性校验

- 随机初始化 A、B，可通过调用开源线性代数库中的 GEMM 接口计算得到正确的结果矩阵，和自己实现的 GEMM 接口的计算结果进行对比，以证明程序实现的正确性，校验不计入测算时间。

3. 计时、规模、线程设置

- 计时和问题规模保持与实验一相同
- 线程数： $T \in \{1, 2, 4, 8, 16\}$ （视机器核数而定，最大不超过机器核数，且为 2 的整数幂，至少取两种不同线程数设置展示性能对比）。

4. 性能指标

- 运行时间
- GFLOPS，以及和理论 GFLOPS 的比值

5. 报告内容

- 将相关实验数据通过图表形式在实验报告中展示并进行分析。

2. 稀疏矩阵向量乘 (SpMV) 的多线程实现

(1) 实验目的

- 结合 CSR 存储与访存特性，设计适合 SpMV 的并行切分与调度，理解带宽受限算子中的负载均衡与访存局部性，掌握稀疏矩阵常用存储格式 CSR。
- 学会利用 Pthreads 的互斥/条件变量实现工作队列或动态调度，缓解非零元分布不均导致的负载倾斜。

(2) 背景知识

与计算密集型的 GEMM 不同，稀疏矩阵向量乘 (SpMV) 是典型的内存带宽受限算子，这意味着程序的运行速度瓶颈不再是 CPU 的浮点计算能力，而是由内存系统获取数据的速度决定，其核心挑战源于稀疏矩阵非零元素的不规则分布，这给并行化带来了与 GEMM 截然不同的问题。

为了高效地存储和计算，我们不能像稠密矩阵那样使用二维数组。CSR 格式是当前最主流的方案之一，它使用三个一维数组来精简地表示一个稀疏矩阵，通过这种设计，第 i 行的所有非零元素及其列索引就紧凑地存储在 `val` 和 `col_idx` 数组的 $[\text{row_ptr}[i], \text{row_ptr}[i+1])$ 这个半开区间内，区间的长度 $\text{row_ptr}[i+1] - \text{row_ptr}[i]$ 直接给出了第 i 行的非零元素个数，这种结构非常适合按行遍历的计算。

基于 CSR 的 SpMV 核心计算循环如下：

```
//SPMV
for (int j = row_ptr[i]; j < row_ptr[i+1]; j++) {
    y[i] += val[j] * x[col_idx[j]];
}
```

可以看到访问 `val[j]` 和 `col_idx[j]` 是流式访问，对缓存友好，但 `x[col_idx[j]]` 却是一次间接内存访问。由于 `col_idx[j]` 的值是任意的，这会导致对输入向量 `x` 的访问模式变得随机、不连续，从而引发大量的缓存未命中，严重制约了访存效率。所以 SpMV 优化的核心就是围绕提升访存局部性与并行负载均衡展开。

并且在稀疏矩阵中，不同行的非零元素个数可能相差巨大，如果像 GEMM 那样简单地按行数平分任务，会导致持有稠密行的线程工作量远超其他线程，造成短板效应，严重降低并行效率。

针对以上问题，大家在使用 Pthreads 并行化 SpMV 时需重点考虑下面几个问题。

首先是线程安全，在 SpMV 的并行实现中，最关键的共享资源是输出向量 y 。如果我们设计的并行方案能保证任意一个 $y[i]$ 在同一时刻只会被一个线程写入，那么对 y 的更新就是天然线程安全的，无需加锁。例如按行块静态切分的策略就满足此条件，因为不同的线程负责计算不同的行 i 。

第二点是同步需求，当采用更高级的动态调度策略来解决负载不均问题时，同步就变得不可或缺，以及互斥量这个问题，如果我们设置一个全局的任务池或共享的行索引计数器，多个线程会去争抢下一个任务，那么对这个共享资源的访问必须由互斥锁保护，以防多个线程取到同一个任务。

最后是条件变量，在生产者-消费者模型（主线程生成任务，工作线程消耗任务）中，如果任务队列为空，工作线程不应空转浪费 CPU，而应使用条件变量进入休眠等待，直到主线程发出有新任务的信号再被唤醒，这是一种高效的线程协作方式。

（3）实验内容

1. 并行实现

- 语言为 C/C++ 语言+Pthreads，数据类型选用 double，实现为 $y = A \times x$ （CSR）。
- 切分策略（建议两种均实现，分析优劣）
 - ✧ 按行块（静态）：将行区间均分给线程；简单但遇到行非零分布不均时易失衡。
 - ✧ 按非零数（近似均匀）：按非零元数量给线程划分任务，负载均衡更好。

2. 稀疏矩阵生成与输入

- 负载均衡矩阵：仍采用和上次实验相同的可调规模和稠密度，随机初始化。
- 负载不均衡矩阵：下节课前会将负载不均衡的矩阵给到大家，并提供读矩阵的源码。

3. 正确性校验

- 可通过调用开源线性代数库中的 SpMV 接口计算得到正确的结果矩阵，和自己实现的并行 SpMV 接口的计算结果进行对比，以证明程序实现的正确性。

4. 计时、规模、线程设置

- 计时和问题规模保持与实验一相同
- 线程数： $T \in \{1, 2, 4, 8, 16\}$ （视机器核数而定，最大不超过机器核数，且为 2 的整数幂，至少取两种不同线程数设置展示性能对比）。

5. 性能指标

- 运行时间
- GFLOPS，以及和理论 GFLOPS 的比值

6. 报告内容

- 将相关实验数据通过图表形式在实验报告中展示并进行分析。

3. Pthreads 多线程实现求 PI 值示例

此示例仅用于同学们学习，不要求开展具体实验。

我们使用经典的蒙特卡洛方法来求 π 作为示例，算法思想是想象一个边长为 2 的正方形，其中心在原点(0,0)，这个正方形的面积是 $2 \times 2 = 4$ ，在这个正方形内部，有一个半径为 1 的内切圆，其面积是 $\pi r^2 = \pi$ ，现在，我们向这个正方形内随机地投掷大量的点，计算落在圆内的点的数量与总投掷点数的比例，这个比例应该约等于圆的面积与正方形面积的比例：

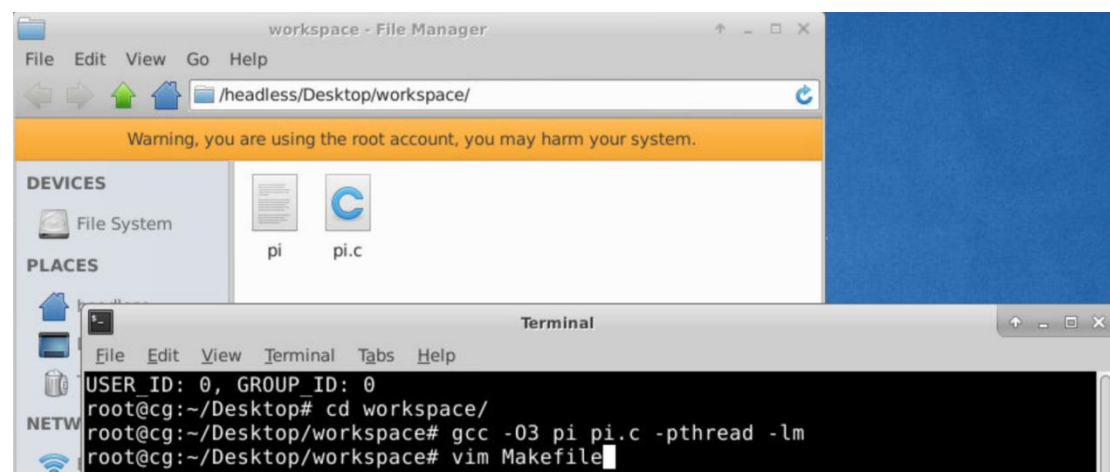
$$\frac{\text{总点数落在圆内的点数}}{\text{总点数}} \approx \frac{\text{圆面积}}{\text{正方形面积}} = \frac{\pi}{4}$$

因此，我们可以估算出 π 的值：

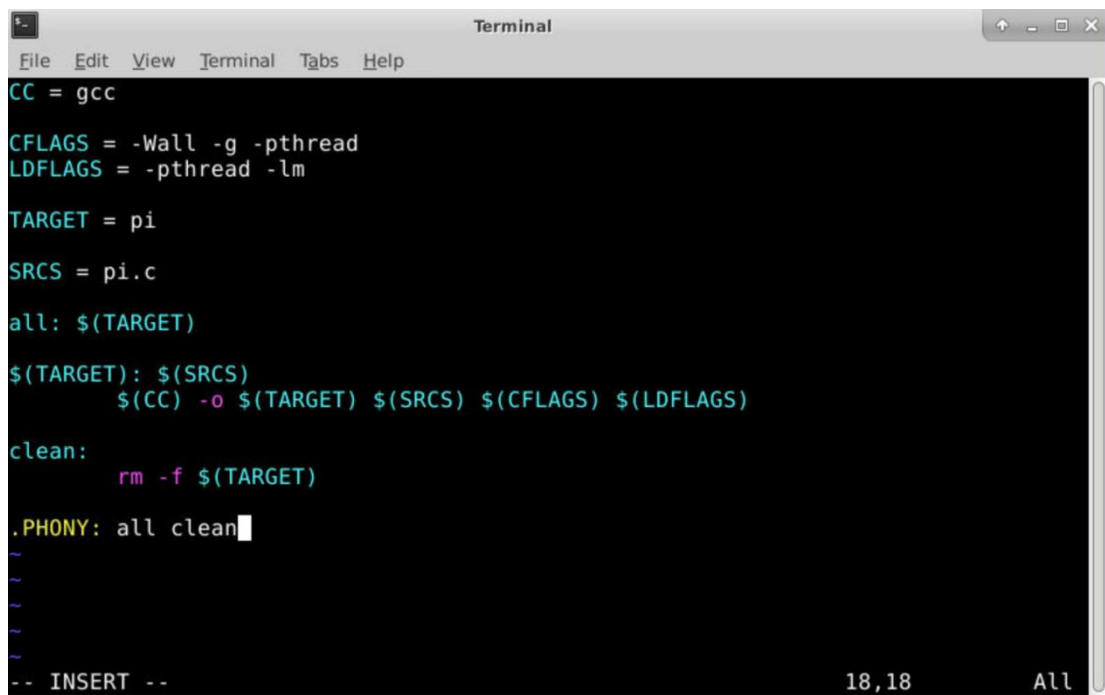
$$\pi \approx 4 \times \frac{\text{总点数落在圆内的点数}}{\text{总点数}}$$

为了并行化，我们可以让多个线程同时进行投点，每个线程负责投掷一部分点。最后将所有线程的结果汇总，即可得到最终的 π 值。

1. 首先，大家将自己编写好的代码放置于 workspace 下，接下来我们有 2 个选择，直接使用 `gcc -o pi pi.c -pthread -lm` 指令编译（推荐大家使用 O3）



2. 为实现编译流程的自动化与标准化，我们引入 Makefile，它是一个强大的构建自动化工具，通过定义文件间的依赖关系和生成规则，能够极大地提升项目管理效率，当项目规模增长，编译指令变得复杂时，Makefile 的可维护性优势尤为突出。这里请大家注意一个关键语法，在 Makefile 中，所有命令必须使用单个 Tab 字符进行缩进，而非空格，这是 make 程序解析语法的强制性规定。

A terminal window titled "Terminal" with a menu bar (File, Edit, View, Terminal, Tabs, Help). It displays the content of a Makefile. The Makefile defines the compiler as gcc, sets CFLAGS to -Wall -g -pthread and LDFLAGS to -pthread -lm, sets the target to pi, and the source to pi.c. It includes rules for building the target and cleaning it. The terminal shows the Makefile content line by line, with a cursor at the end of the .PHONY: all clean line.

```
CC = gcc

CFLAGS = -Wall -g -pthread
LDFLAGS = -pthread -lm

TARGET = pi
SRCS = pi.c

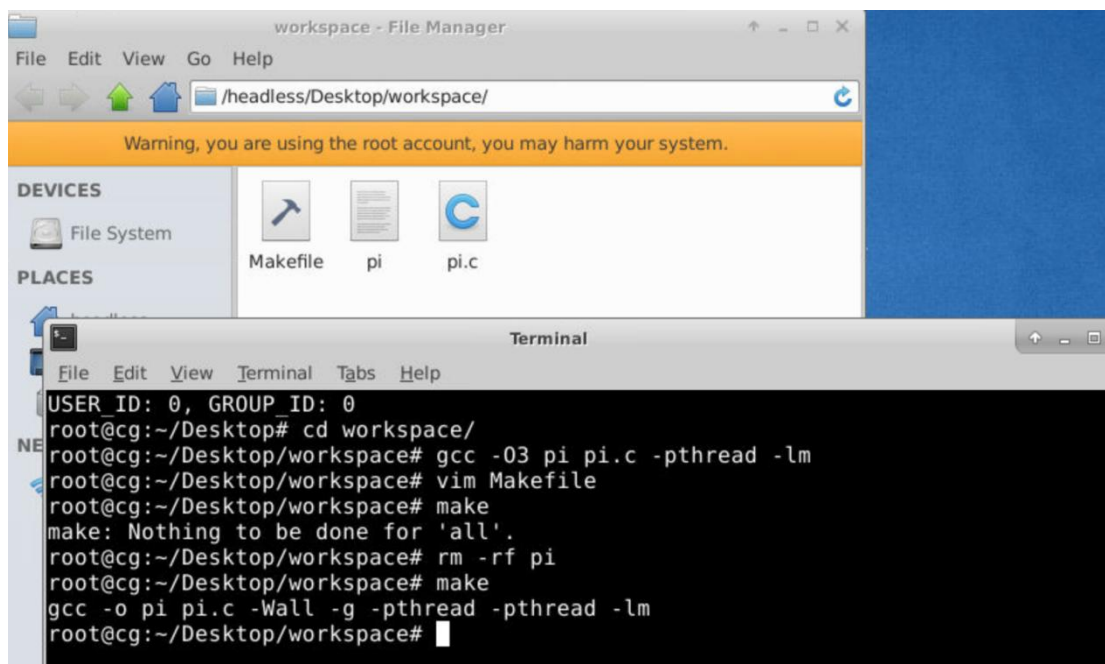
all: $(TARGET)

$(TARGET): $(SRCS)
    $(CC) -o $(TARGET) $(SRCS) $(CFLAGS) $(LDFLAGS)

clean:
    rm -f $(TARGET)

.PHONY: all clean
```

3. 然后 make 之后就可以生成二进制文件了，一般情况下，为了规范我们将其放置在 obj 文件夹下，方便后续维护管理。

A screenshot showing a File Manager window titled "workspace - File Manager" and a terminal window. The File Manager shows the directory /headless/Desktop/workspace/ with files Makefile, pi, and pi.c. The terminal shows the execution of commands to compile the program. The terminal output shows that the program is already built, so 'make' does nothing. Then, the program is removed with 'rm -rf pi' and rebuilt with 'make', which successfully compiles the program using the specified flags.

```
workspace - File Manager
File Edit View Go Help
/headless/Desktop/workspace/
Warning, you are using the root account, you may harm your system.
DEVICES
File System
PLACES
Makefile pi pi.c

Terminal
File Edit View Terminal Tabs Help
USER_ID: 0, GROUP_ID: 0
root@cg:~/Desktop# cd workspace/
root@cg:~/Desktop/workspace# gcc -o3 pi pi.c -pthread -lm
root@cg:~/Desktop/workspace# vim Makefile
root@cg:~/Desktop/workspace# make
make: Nothing to be done for 'all'.
root@cg:~/Desktop/workspace# rm -rf pi
root@cg:~/Desktop/workspace# make
gcc -o pi pi.c -Wall -g -pthread -pthread -lm
root@cg:~/Desktop/workspace#
```

4. 接下来我们直接运行生成的二进制文件就可以得到结果。

```

root@cg:~/Desktop/workspace# make clean
rm -f pi
root@cg:~/Desktop/workspace# make
gcc -o pi pi.c -Wall -g -pthread -pthread -lm
root@cg:~/Desktop/workspace# ./pi
使用了 8 个线程
总投点数：100000000，每个线程投点数：12500000

计算完成。
落在圆内的总点数：78535450
Pi ( $\pi$ ) 的估算值：3.141418
root@cg:~/Desktop/workspace#

```

5. Pthread 代码解读

```

1  #include <stdio.h> //主线程分发任务 -> 各子线程并行处理 -> 主线程等待并回收所有结果 -> 主线程计算最终答案
2  #include <stdlib.h>
3  #include <pthread.h> // POSIX线程库API
4  #include <time.h>
5  #define THREAD_COUNT 8 // 编译时常量，我们用来定义线程池大小
6  #define TOTAL_POINTS 100000000 //编译时常量，我们自己定义模拟的总样本点数
7  typedef struct {
8      long long points_to_check; // 线程需处理的样本点数量
9      long long* result_hits; // 指向主线程内存，用于回写该线程的计算结果
10 } ThreadData;
11 void* thread_task(void* arg) {
12     ThreadData* data = (ThreadData*)arg;
13     long long hits = 0;
14     unsigned int seed = time(NULL) ^ (unsigned int)pthread_self(); // 使用时间和线程ID生成唯一的随机数种子，确保线程间的随机序列不同
15     for (long long i = 0; i < data->points_to_check; ++i) {
16         double x = (double)rand_r(&seed) / RAND_MAX * 2.0 - 1.0; // rand_r是rand的可重入（线程安全）版本
17         double y = (double)rand_r(&seed) / RAND_MAX * 2.0 - 1.0;
18         if (x * x + y * y <= 1.0) {
19             hits++;
20         }
21     }
22     *(data->result_hits) = hits; // 将局部计算结果写入主线程指定的内存位置
23     return NULL;
24 }
25 int main() {
26     pthread_t thread_ids[THREAD_COUNT]; // 1. 资源初始化
27     ThreadData thread_args[THREAD_COUNT];
28     long long thread_results[THREAD_COUNT] = {0};
29     long long total_hits = 0;
30     long long points_per_thread = TOTAL_POINTS / THREAD_COUNT;
31     printf("使用了 %d 个线程\n", THREAD_COUNT);
32     printf("总投点数: %lld, 每个线程投点数: %lld\n", (long long)TOTAL_POINTS, points_per_thread);
33     for (int i = 0; i < THREAD_COUNT; ++i) { // 2. 线程创建与分发 (Fork)
34         thread_args[i].points_to_check = points_per_thread;
35         thread_args[i].result_hits = &thread_results[i];
36         int status = pthread_create(&thread_ids[i], NULL, thread_task, (void*)&thread_args[i]);
37         if (status) {
38             printf("创建线程失败，错误码: %d\n", status);
39             exit(-1);
40         }
41     }
42     for (int i = 0; i < THREAD_COUNT; ++i) { // 3. 线程同步与结果聚合 (Join & Reduce)
43         pthread_join(thread_ids[i], NULL);
44         total_hits += thread_results[i];
45     }
46     double pi = 4.0 * total_hits / TOTAL_POINTS; // 4. 最终计算与输出
47     printf("\n计算完成.\n");
48     printf("落在圆内的总点数: %lld\n", total_hits);
49     printf("Pi ( $\pi$ ) 的估算值: %f\n", pi);
50     return 0;
51 }

```